DocuSynth Reference Document (Synthetic)

This document describes a Dockerized document intelligence platform.
It uses FastAPI for the control plane and Python for retrieval-augmented
generation. PostgreSQL with the pgvector extension stores both document
chunks and a semantic cache of normalized queries with their embeddings.
Redis provides exact-match query caching and per-user rate limiting.
JWT tokens authenticate requests against protected endpoints.

Architecture overview:
Clients send authenticated requests to /api/v1/query. The control plane first checks Redis for an exact match using a normalized hash of the question. On a miss it embeds the question via the RAG service, then performs a cosine-distance lookup in the pgvector semantic_cache table. If similarity exceeds the configured threshold (default 0.85) the cached answer is returned. Otherwise retrieval, council generation, peer review, and chairman synthesis run sequentially before storing results in both caches.

Observability and operations:
Prometheus scrapes /metrics on the backend and Grafana visualizes request rate, latency percentiles, cache hit rate, retrieval and embedding latency, LLM call counts, and error counts. Structured tagged logs ([HTTP], [Council], [Cache], [Audit], [Error], [RAG]) carry user_id, doc_id, query hashes, and per-stage timings for forensic analysis. Deployment is orchestrated through Docker Compose for local development with parity to a future Kubernetes deployment.

Failure handling and trade-offs:
If retrieval fails the request returns 503 and increments a labelled error counter. If all council members fail synthesis falls back to the single best candidate with reduced confidence. Trade-offs include additional latency from peer review in exchange for higher answer robustness, and embedding overhead in exchange for paraphrase-tolerant caching. Future improvements include async ingestion via a Redis worker and fine-grained per-document semantic-cache namespacing.